

Reactive Data Streams

1.0 Introduction

Modern interactive systems process continuously changing data from many independent sources. Game engines, simulations, distributed systems, and user interfaces all share a common requirement: changes must be propagated efficiently and deterministically to interested consumers.

Traditional event-based systems often rely on direct relationships between producers and consumers. While simple, these systems become difficult to manage as the number of dependencies grows. Reactive programming addresses this problem by representing changing values as streams of data that can be observed and transformed. Rd (Reactive Data Streams) builds upon this idea but introduces a different architectural model. A reactive data stream in Rd is not merely a chain of operations. It is a complete data flow unit composed of a source, optional mutators, and an observable node.

A stream represents the ownership boundary of a data flow. The observable defines how external consumers interact with the stream, while internal processing remains composed from specialized execution components.

The central design principle of Rd is that behavior belongs to the nodes that own the data flow. The system itself does not decide globally how all streams should execute. Instead, each node determines its own processing and dispatch behavior through explicit strategies.

This approach is comparable to the role of a system bus in an operating system kernel. A system bus provides a defined communication boundary between components without requiring the entire system to understand the internal behavior of every connected device. Similarly, a reactive data stream provides a controlled communication boundary while allowing each node to define its own execution characteristics

1.1 Reactive Programming

Reactive programming is a programming paradigm focused on representing changing values and asynchronous data sources as streams of information. It is based on the observer pattern: Instead of explicitly requesting values at specific points in time, consumers subscribe to changes and react when new data becomes available. This allows systems to model continuously evolving state, events, and external inputs in a unified way.

A reactive system typically consists of:

- Data producers that generate values over time
- Consumers that observe and react to changes
- Operations that transform or filter incoming data

The fundamental idea is that data flow becomes explicit. Dependencies are represented as relationships between sources and consumers instead of being hidden inside control flow.

Reactive programming itself does not prescribe a specific implementation model. It describes a way of reasoning about changing data and the propagation of

updates. Different frameworks and systems can implement reactive programming using different approaches, execution models, and memory strategies

1.2 ReactiveX (Rx)

ReactiveX is a family of libraries that provide reactive programming abstractions through observable sequences. The central concept of Rx is the observable sequence. A producer exposes an observable object, and consumers subscribe to receive notifications when values are emitted. Additional operators such as filtering, mapping, merging, and scheduling create derived observable sequences.

This model provides a powerful and expressive way to compose asynchronous data flows. Operator chains can describe complex transformations while keeping application code concise. However, the abstraction model of Rx also introduces specific design characteristics. Operators are commonly represented as layers in a runtime chain, and execution behavior such as scheduling is often modeled through separate scheduler abstractions applied to observable sequences

1.3 Reactive Programming isn't Rx

Reactive programming and Rx are closely related concepts, but they are not identical. Reactive programming describes a general programming paradigm. ReactiveX is a specific family of implementations and APIs based on this paradigm.

A system can implement reactive programming without following the design decisions of ReactiveX. Concepts such as observable sequences, operators, schedulers, and subscriptions are common

in Rx implementations, but they are not fundamental requirements of reactive programming itself.

Rd follows the principles of reactive programming while intentionally choosing a different execution model. Instead of treating every operation as a runtime operator in a potentially large chain, Rd separates data processing, stream materialization, dispatching, and subscription management into independent responsibilities. This allows Rd to preserve the expressive nature of reactive data processing while providing explicit control over execution behavior

2.0 Reactive Streaming Model

The fundamental unit of Rd is the reactive data stream. This concept represents a complete flow of data from its origin to its consumers. Unlike models where individual operators form the primary abstraction, Rd considers the complete stream definition as the central unit.

A reactive data stream consists of three conceptual components:

- **Sources** define where data enters the stream
- **Mutators** describe optional processing steps applied to incoming data
- **Observables** represent the materialized boundary through which external consumers can subscribe to the stream

A stream does not require every intermediate processing step to become a runtime object. Instead, the stream description can be composed and optimized

before an observable boundary is created. This allows Rd to separate the description of data processing from the runtime representation required for execution

2.1 Source

A Source is the entry point of a reactive data stream. It represents a component capable of producing values and introducing them into the reactive system. A Source does not define how values are consumed or processed. Its responsibility is limited to providing a stable origin for a data flow

A Source may exist without an Observable. This allows data processing to be composed before a stream becomes visible to external consumers. The separation between Source and Observable is intentional. A producer does not need to know how its data will eventually be consumed

2.2 Mutators

Mutators define how data inside a reactive data stream is processed. A Mutator does not represent a standalone runtime object in the stream. Instead, it describes an accumulated processing step that becomes part of the stream execution behavior.

The responsibility of a Mutator is limited to transforming or controlling the propagation of an individual value. A Mutator receives an input value and determines whether a resulting value should continue through the stream.

This allows different kinds of processing to be expressed:

- Gating operations that keep the same data representation

- Projection operations that create a new data representation

The first operation controls whether data continues, while the second changes the representation of the data. Mutators are intentionally independent from execution concerns. They do not define: which thread executes the operation, when the result is dispatched, how observers are stored or whether processing occurs synchronously or asynchronously. These responsibilities belong to the stream node and its strategies

2.3 Observable

An Observable is the materialized boundary of a reactive data stream. While a Source defines where data originates and Mutators define how values are processed, the Observable defines how external components interact with the stream.

An Observable implements the observer pattern and provides the connection between internal stream processing and external consumers. It does not need to know the details of the processing that happened before it. It only receives the resulting data contract defined by the stream.

This separation allows the same processing logic to be used with different execution models, the same stream definition can be materialized with different node behaviors. The data processing remains unchanged while the runtime behavior of the stream can vary

2.4 Stream Ownership Model

A core design principle of Rd is explicit ownership of behavior. A reactive data stream owns the decisions required to process and propagate its data. There is no

global system policy that determines how every stream must execute.

Each observable node defines its own behavior through composition:

- Mutators define value processing
- Dispatching Strategies define propagation behavior
- Subscription Strategies define observer management

This model allows different parts of a system to use different execution characteristics without requiring changes to the overall reactive infrastructure. For example, a gameplay stream may require immediate synchronous dispatch, while a rendering-related stream may use buffered or parallel processing.

Both streams follow the same reactive data model while maintaining independent execution policies. The result is a decentralized reactive system where behavior is defined locally at the point where data ownership exists

3.0 Stream Composition and Materialization

Reactive Data Streams in Rd are composed incrementally. A Source provides the initial data type of a stream. Additional operations can modify how values are processed before the stream becomes observable.

These operations do not immediately create independent runtime objects. Instead, they extend the stream description by adding processing semantics.

The individual operations define the behavior of the stream, but they do not

define the runtime structure by themselves. This distinction allows Rd to separate stream composition from stream execution

3.1 Mutator Composition

A Mutator describes how an individual value is processed inside a reactive data stream. Mutators are composed according to their data contract rather than their individual operation type. The relevant distinction is whether an operation preserves the current data representation or creates a new one.

Rd therefore distinguishes between two fundamental stream states

Gating Stream

A gating stream keeps the same input and output representation. Operations in this category control whether a value continues through the stream but do not change its type:

- Filtering
- Validation
- Conditional propagation

A gating mutator can decide that a value should not continue or allow the value to be forwarded unchanged

Projection Stream

A projection stream changes the data representation from *TIn* to *TOut*. Projection streams therefore create a new output representation from an input value:

- Mapping data to another type
- Extracting a component
- Creating a derived representation

A projection mutator produces a new value, while a projection operation may combine

transformation and conditional propagation without requiring additional runtime stages

3.2 Stream State Transition

A gating stream may transition into a projection stream when an operation changes the data representation:

```
Source<Entity>
|
Where(IsActive)
|
Select(GetTransform)
```

After this transition, the stream remains a projection stream. Further operations are composed inside this representation:

```
Entity
|
Select(GetTransform)
|
Where(IsVisible)
|
Select(CreateRenderData)
```

The internal processing may contain multiple filtering and transformation steps, but the observable contract remains stable to create a monotonic type progression.

A projection stream does not transition back into a gating stream because the original data representation is no longer the externally visible output of the stream

3.3 Mutator Materialization

Before a stream becomes observable, its composed operations are materialized into a concrete execution representation. Materialization converts the logical stream description into specialized structures optimized for execution.

```
Source
|
Where Object
|
Select Object
|
Where Object
|
Observable
```

From the above, Rd creates a compact representation over a specialized Mutator. The resulting Mutator contains the complete processing behavior required for the stream. This allows the runtime representation to avoid unnecessary intermediate allocations and runtime dispatch between individual operations. The exact representation however depends on the stream contract

3.4 Pipeline Fusion

The composition model enables Rd to combine multiple operations into a single execution unit, that does not require every operation to exist as an independent runtime layer.

Instead, the stream can be represented as one specialized mutator that preserves the complete behavior: The result is a single execution path where the runtime only

needs to process the materialized stream representation.

While this approach differs from models where every operator remains a separate runtime object and therefore allows for each step to accept branching, Rd treats composition as a step that produces an optimized stream representation. If additional branching is required, the stream must explicitly be branched at the desired outcome instead

4.0 Observable Nodes and Local Responsibility

A fundamental design principle of Rd is that responsibility is assigned locally to the node that owns a data flow boundary. Traditional reactive systems often rely on global mechanisms to control execution behavior. Scheduling, threading, buffering, and subscription management are commonly provided by shared infrastructure that applies common rules across all streams.

Rd follows a different approach. A reactive data stream is responsible for defining its own execution behavior through the composition of its observable node. The system provides the mechanisms required to build and connect streams, but individual nodes decide how their own data is processed and propagated. This creates a decentralized execution model where behavior is determined by ownership rather than by global policy.

The observable node forms the boundary where a stream becomes part of the wider system

4.1 Observable Nodes as Data Boundaries

An Observable Node represents a materialized connection between a reactive data stream and its consumers. The node owns the policies required to expose the stream externally while keeping the internal processing model independent.

An Observable Node is responsible for:

- Maintaining subscriptions
- Receiving processed values from the stream
- Applying dispatch behavior
- Forwarding values to observers

The node does not need knowledge about the origin of the data or the individual operations that produced it. From the perspective of a consumer, the node provides a stable observable contract:

Reactive Data Stream

|

v

Observable Node

|

v

Observer API

This allows producers and consumers to evolve independently

4.2 Local Execution Policies

Execution behavior in Rd is not a property of the entire reactive system. Instead, each

Observable Node defines its own execution characteristics. A node can decide: whether values are dispatched immediately, whether values are processed in batches, whether dispatch occurs asynchronously, whether processing is parallelized.

These decisions are represented through explicit strategies. The separation of responsibilities can be described as:

- **Mutator** – defines: "What happens to a value?"
- **Dispatching Strategy** – defines: "How and when is a value propagated?"
- **Subscription Strategy** – defines: "How are observers managed?"

No individual component is required to understand the complete system. Each component only owns the decisions relevant to its responsibility

4.3 Observable Nodes as System Components

The decentralized model of Rd allows individual streams to behave differently within the same application. For example, a gameplay state stream may require immediate synchronous updates, while a telemetry stream may prefer buffered processing.

Both streams use the same reactive data model but have different local execution behavior. The system does not need to distinguish between these cases globally. The difference exists only at the node that owns the respective stream.

The responsibility model of Rd is comparable to the role of a system bus in

operating systems. A system bus provides a communication mechanism between independent components. It defines how components exchange information, but it does not dictate the internal implementation of each connected device.

Similarly, Rd defines a common communication model between reactive components while allowing each observable node to define its own behavior

4.4 Implications for Large Systems

The local responsibility model allows Rd to scale across complex systems where different data flows have different requirements. In a game engine environment, different subsystems may require different stream characteristics.

A global reactive policy would force all of these systems into the same execution model. Rd instead allows each data flow boundary to choose the behavior appropriate for its context while maintaining a unified reactive programming model.

This results in a system where complexity is distributed according to ownership rather than centralized inside the framework

5.0 Memory and Performance Model

Rd is designed around the principle that performance characteristics should emerge from the data model itself rather than from additional optimization layers. Reactive Data Streams are not represented as chains of runtime objects. Instead, stream composition produces a specialized representation that contains the required execution behavior.

This reduces unnecessary intermediate allocations and allows the runtime representation to remain close to the actual data flow

5.1 Value-Oriented Processing

Mutators in Rd are designed as lightweight processing units. Rather than allocating objects for every transformation step, stream composition can produce compact representations that are optimized for direct execution:

- Reducing allocation pressure
- Better cache behavior
- More predictable memory usage
- Improved optimization opportunities for managed and native environments

The resulting execution model is based on processing values directly instead of traversing large object graphs. This is beneficial explicitly for performance critical systems like a game engine

5.2 Optimization Through Specialization

Rd uses type information as part of its execution model. Different stream representations can be specialized depending on their data contract. This allows the runtime to avoid unnecessary abstraction layers while preserving a consistent reactive programming model.

In environments with advanced compiler optimization, such as the .NET CLR/JIT or native template-based compilation, these specialized representations can be optimized into efficient execution paths.

5.3 Controlled Memory Ownership

A reactive system used in dynamic environments requires predictable memory behavior. Rd avoids making memory management a hidden global concern. Instead, memory-related decisions belong to the components that own the corresponding data boundary.

This allows different streams to choose representations suitable for their context while maintaining the same reactive interface. The result is a model where flexibility and performance are not opposing goals, because execution behavior remains explicit and locally controlled

6.0 Conclusion

Rd defines a reactive data model focused on explicit ownership, composability, and predictable execution behavior, specialized for performance critical environments and highly dynamic systems.

Instead of treating reactive programming as a chain of runtime operators, Rd models a reactive data stream as a complete unit consisting of a Source, optional Mutators, and an Observable node. This allows data processing and execution behavior to be composed independently while keeping the runtime representation compact and deterministic.

The central design principle of Rd is that responsibility belongs to the component that owns the corresponding data boundary. Mutators define how values are processed, Observable nodes define how streams become consumable, and strategies define the local execution behavior of each node.

This approach allows reactive data streams to be used in environments where different

systems have different requirements regarding latency, throughput, memory usage, and execution models. Instead of enforcing a global policy, Rd enables each stream to define the behavior appropriate for its context